

检查点06 阶段总结报告 (Version 01)

YOLOv0 的Backbone 演化、完整Loss 升级与评估补充

2026-04-19

Contents

1	实验目标与本报告覆盖范围	3
2	工程组织、数据来源与统一训练协议	3
2.1	工程目录	3
2.2	数据集与输入预处理	4
2.3	统一训练协议	4
3	第一阶段：在简化MSE loss 下进行Backbone 对比	5
3.1	阶段目标	5
3.2	控制变量与结构差异	5
3.3	Backbone 对比结果	6
3.4	第一阶段结论与转移原因	6
4	第二阶段：Stage A，在deep_cnn 上升级单框完整YOLO-style loss	7
4.1	为什么要升级loss	7
4.2	Stage A 的loss 设计	7
4.3	Stage A 结果	8
5	第三阶段：Stage B，在相同完整单框loss 下比较残差连接	8
5.1	为什么进入Stage B	8
5.2	Stage B 配置	8
5.3	Stage B 结果	9
6	第四阶段：Stage C，三框检测头与三框完整loss	9
6.1	为什么从单框转向三框	9
6.2	Stage C 的网络结构、深度与张量形状	10
6.2.1	Backbone: deep_residual	10
6.2.2	Stage C 的逐层张量流	10
6.2.3	检测头与输出	11
6.3	Stage C 的检测方法、anchors 与target 编码	11
6.3.1	三组anchors	11
6.3.2	target 分配方法	11
6.3.3	预测解码与推理	12
6.4	Stage C 的loss 设计	12
6.5	Stage C 正式超参数	12
6.6	Stage C 结果	13
6.7	Stage C 结论与转移原因	14

7	第五阶段：检测评估补充与COCO 子集官方风格评测	14
7.1	为什么需要评估补充	14
7.2	为什么官方风格评测只在COCO 子集上进行	15
7.3	Stage A/B/C 在COCO 子集上的结果	15
7.4	评估补充阶段结论	16
8	当前loss 设计、网络演化方向与阶段性总括	16
8.1	已经完成的演化路径	16
8.2	loss 设计从早期版本到当前版本的变化	17
8.3	当前最优实用系统	17
8.4	当前网络演化方向	17
9	总结	17

1 实验目标与本报告覆盖范围

本报告总结yolov0 在检查点6 中截至目前已经完成的工作。当前工作的总体目标不是立即得到一个已经成熟的工业级检测器，而是把此前的教学式MiniYOLO 原型推进为一个更正式、更可复现、也更适合做结构对比与损失对比的小型检测工程基线。

截至本报告写作时，已经完成的主要内容包括：

1. 在早期简化MSE loss 下完成一轮严格控制变量的Backbone 对比实验；
2. 在固定backbone 的前提下，将损失函数从教学式 $\text{MSE}_{\text{cls}} + \text{MSE}_{\text{box}}$ 升级为更接近实际检测器的单框YOLO-style 完整loss；
3. 在相同完整loss 下比较残差连接的作用；
4. 实现并完成第一版三框检测头与三框完整loss 的full 训练；
5. 补充更正式的检测评估，包括COCO 子集官方风格评测、FLOPs 估计和FPS 测量。

本报告严格只记录已经完成的内容，不包含尚未落地的后续改进。整体按阶段行文：每一组实验放在同一部分中描述，并在每一部分结尾明确说明为什么要转向下一部分。

2 工程组织、数据来源与统一训练协议

2.1 工程目录

检查点6 的全部开发都位于：

- /home/lidz/YOLO/yolov0

其中与本报告直接相关的关键目录如下：

- configs/：正式训练配置；
- models/：Backbone、Head 与Detector；
- data/：数据集与target 编码；
- losses/：损失函数；
- engine/：训练与验证循环；
- tools/：训练、评估、FLOPs/FPS 测量、图表导出；
- logs/records/：每次正式实验的配置快照、结果文本、元数据；
- outputs/：权重、预测可视化和图表；
- runs/：TensorBoard 事件文件；
- report/：本报告。

2.2 数据集与输入预处理

当前全部正式实验使用Unified 的统一manifest:

- 训练集: `/home/lidz/YOLO/DataSet/Unified/manifests/all_train.jsonl`
- 验证集: `/home/lidz/YOLO/DataSet/Unified/manifests/all_val.jsonl`

输入处理流程为:

- 从manifest 中读取图像路径与标注;
- 用PIL 读图并统一转为RGB;
- resize 到320 x 320;
- 同步缩放xyxy 真实框;
- 编码成YOLO 风格的监督张量。

当前阶段不使用随机增广。这一选择是有意的, 因为检查点6 前半段的重点是Backbone 与loss 的受控比较。如果此时引入增广, 会额外增加一个大变量, 弱化结构对比的解释力。

2.3 统一训练协议

在报告中涉及的正式实验里, 以下训练设置尽量保持一致:

Table 1: 当前正式实验共享的训练协议

项目	当前设置
训练manifest	<code>all_train.jsonl</code>
验证manifest	<code>all_val.jsonl</code>
打包格式	<code>pt</code>
输入尺寸	<code>320 x 320</code>
输出网格	<code>10 x 10</code>
类别数	<code>80</code>
batch size	<code>256</code>
epochs	<code>20</code>
优化器	<code>AdamW</code>
学习率	<code>1e-3</code>
最小学习率	<code>1e-5</code>
权重衰减	<code>1e-4</code>
调度器	<code>cosine</code>
DataLoader workers	<code>64</code>
多卡方式	<code>DataParallel</code>
实验类型	<code>full training only</code>

这里只接受full 训练结果。smoke test 只用于shape、语法和loss sanity check, 不进入正式对比表, 也不进入阶段结论。

3 第一阶段：在简化MSE loss 下进行Backbone 对比

3.1 阶段目标

第一阶段的目标是先冻结输入尺寸、网格、优化器和数据协议，只比较Backbone 结构本身。此时检测器仍然采用比较早期的简化形式：

- 每个grid cell 只预测一个框；
- 输出张量为[B, 10, 10, 84]；
- 其中84 = 80 classes + 4 box values；
- 损失函数仍然是分类MSE + 框回归MSE，没有objectness，也没有IoU 型box loss。

这一阶段完成了四个正式Backbone 实验：

- baseline_cnn
- deep_cnn
- residual_small
- resnet18_like

3.2 控制变量与结构差异

四个实验共用表1 中的训练设置，因此主要变化来自Backbone：

- baseline_cnn：基础深度，不加残差；
- deep_cnn：只增加CNN 深度，不加残差；
- residual_small：保持baseline 深度，只增加轻量残差块；
- resnet18_like：切换到更标准的ResNet-18 风格残差主干。

这里有一个重要说明：

residual_small 不是deep_cnn + residual，而是baseline_cnn + residual。

因此这一阶段实际回答的是：

- 单纯加深CNN 是否有效；
- 在baseline 深度下加入残差是否有效；

但它还不能回答：

- 更深的网络再配残差是否更好。

3.3 Backbone 对比结果

Table 2: 简化MSE loss 下的Backbone 正式对比结果

Model	Params	Avg epoch time (s)	Best val loss	Best epoch	Final val loss
baseline_cnn	8.72M	168.47	0.208655	13	0.242123
deep_cnn	17.66M	183.19	0.201727	14	0.223911
residual_small	17.57M	182.76	0.204629	10	0.238338
resnet18_like	12.38M	172.75	0.223215	9	0.270369

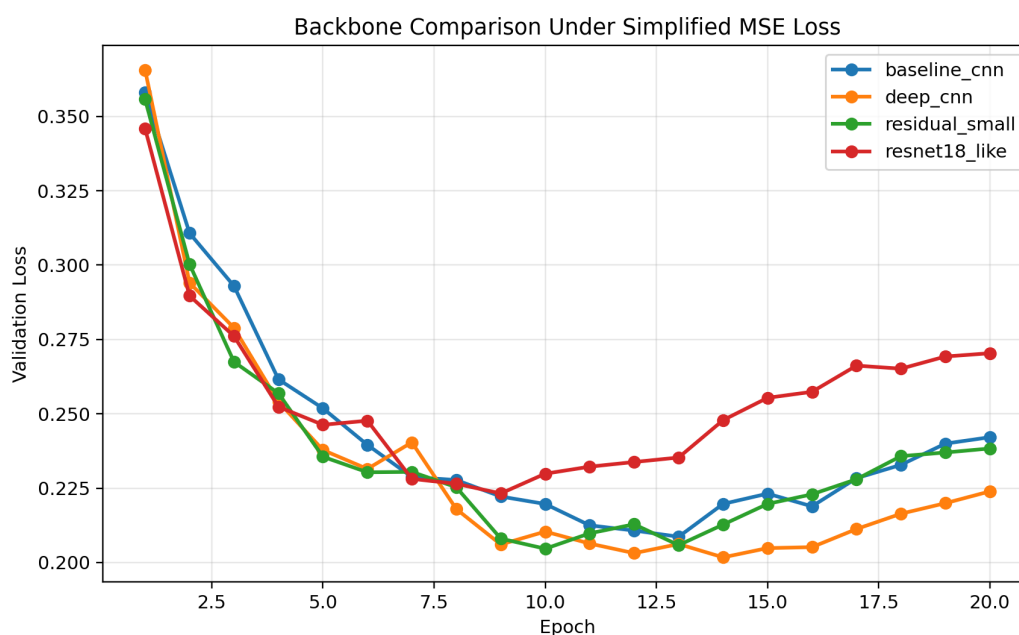


Figure 1: 四个Backbone 在简化MSE loss 下的验证集损失曲线。该图由正式实验的result.txt 历史记录导出。

3.4 第一阶段结论与转移原因

这一阶段的结论是：

- deep_cnn 取得了四个Backbone 中最好的验证表现；
- residual_small 优于baseline_cnn，但没有超过deep_cnn；
- resnet18_like 结构更规范，但在当前简化训练目标下泛化较差；
- resnet18_like 的train-val 分化最明显，因此被记录为更强的过拟合/泛化风险信号。

因此，第一阶段收口时的决策是：

先冻结Backbone 线，把deep_cnn 作为后续loss 升级的主干基线；不要继续盲目堆叠新的Backbone，而是先把训练目标做对。

4 第二阶段：Stage A，在deep_cnn 上升级单框完整YOLO-style loss

4.1 为什么要升级loss

在第一阶段中，检测头输出为：

$$[B, 10, 10, 84]$$

对应：

- 4 个框参数；
- 80 个类别值；
- 没有objectness；
- 没有no-object 约束；
- 没有IoU 型box 几何监督。

Stage A 的核心任务是把这一设置升级为更接近真实检测器的单框YOLO-style loss。主要改动包括：

- 增加objectness 分支；
- 用GIoU 替换框回归MSE；
- 分类项改用BCEWithLogits + one-hot；
- 显式拆分loss_box、loss_obj 和loss_cls。

因此输出形状变为：

$$[B, 10, 10, 85]$$

每个cell 的预测向量顺序为：

$$[t_x, t_y, t_w, t_h, obj, cls_1, \dots, cls_{80}]$$

4.2 Stage A 的loss 设计

Stage A 使用的损失为：

$$L_{box} = 1 - \text{GIoU} \quad (1)$$

$$L_{obj} = L_{obj}^+ + \lambda_{noobj} L_{obj}^- \quad (2)$$

$$L_{cls} = \text{BCEWithLogits}(cls, target) \quad (3)$$

$$L = \lambda_{box} L_{box} + \lambda_{obj} L_{obj} + \lambda_{cls} L_{cls} \quad (4)$$

这里已经固定下来的关键设计是：

- 分类项：BCEWithLogits + one-hot target
- t_x, t_y, t_w, t_h：第一版统一经过sigmoid
- target_obj：硬标签1/0

- `lambda_noobj = 0.5`

权重设置为:

$$\lambda_{box} = 5.0, \quad \lambda_{obj} = 1.0, \quad \lambda_{noobj} = 0.5, \quad \lambda_{cls} = 1.0$$

4.3 Stage A 结果

Table 3: Stage A: `deep_cnn + single-box full loss`

项目	数值
Run ID	<code>deep_cnn_single_box_full_loss_20260419_020616</code>
Params	17,664,437
输出张量形状	(1, 10, 10, 85)
最优epoch	14
最优验证总损失	2.391537
最终训练总损失	2.163206
最终验证总损失	2.439510

这一阶段说明:

- 完整单框YOLO-style loss 已经在full 训练上稳定跑通;
- `loss_box`、`loss_obj`、`loss_cls` 均能正常下降;
- `mean_iou` 随训练持续上升;
- 项目已经从教学式的MSE 检测训练进入更真实的检测训练阶段。

5 第三阶段: Stage B, 在相同完整单框loss 下比较残差连接

5.1 为什么进入Stage B

Stage A 已经验证了完整单框YOLO-style loss 是可训练且稳定的。接下来最干净比较方式是:

保持loss 不变, 只改变Backbone。

因此Stage B 的问题很明确:

在相同的完整单框loss 下, 残差连接是否能比纯`deep_cnn` 带来更好的检测训练效果?

5.2 Stage B 配置

Stage B 采用:

- Backbone: `deep_residual`
- 输出: [B, 10, 10, 85]
- loss: 与Stage A 完全相同
- 训练协议: 与Stage A 完全相同

因此Stage A 和Stage B 唯一实质性的结构差异是：

- A: deep_cnn
- B: deep_residual

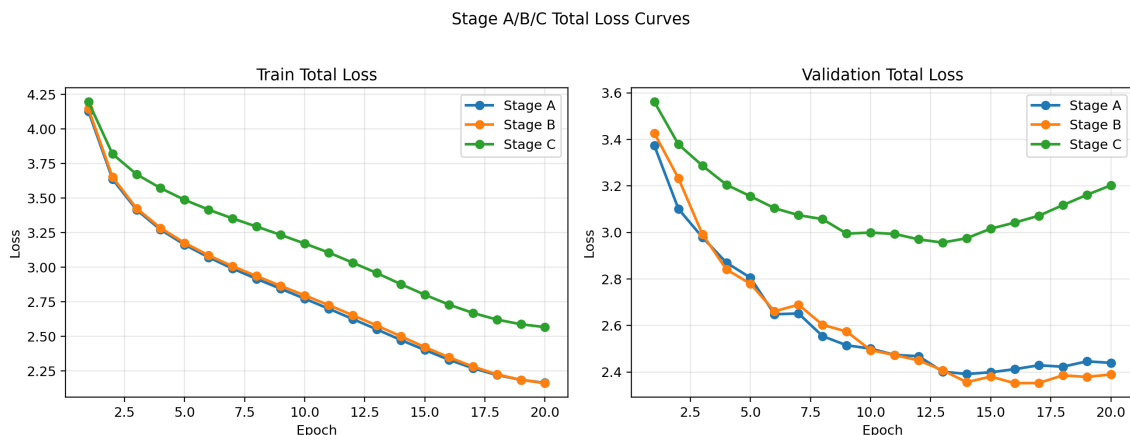


Figure 2: Stage A、Stage B、Stage C 的训练与验证总损失曲线。

5.3 Stage B 结果

Table 4: Stage B: deep_residual + single-box full loss

项目	数值
Run ID	deep_residual_single_box_full_loss.20260419_031404
Params	26,516,917
输出张量形状	(1, 10, 10, 85)
最优epoch	16
最优验证总损失	2.351891
最终训练总损失	2.160557
最终验证总损失	2.389557

Stage B 优于Stage A。这一点非常重要，因为它说明：

- 在更完整的检测训练目标下，残差连接的收益比在第一阶段更明显；
- loss 设计的升级会改变模型排序，因此不能简单地把第一阶段的Backbone 结论外推到完整检测训练上；
- 当前最优的实用baseline 由此变成了：

deep_residual + single-box full loss

6 第四阶段：Stage C，三框检测头与三框完整loss

6.1 为什么从单框转向三框

Stage B 已经得到一个稳定的单框最佳基线。下一步自然的改动方向是：

在不改变核心Backbone 的前提下，把检测头从单框升级为三框。

这样做的动机主要有三点:

- 缓解单框单cell 对多目标与不同尺度目标表达能力有限的问题;
- 引入anchor 匹配, 使detector head 更接近真实YOLO 风格;
- 为后续更复杂的系统级升级打基础。

6.2 Stage C 的网络结构、深度与张量形状

Stage C 是当前已经完成的最完整检测系统, 因此这里对它做重点展开。

6.2.1 Backbone: deep_residual

Stage C 的Backbone 属于BaselineBackbone 家族, 通道设置为:

32, 64, 128, 256, 384, 512

在 $\text{depth_mult} = 2.0$ 下, 每个stage 含有4 个ConvBNAct block; 对于stage index ≥ 2 , 再额外插入一个轻量ResidualBlock。如果按卷积层数粗略统计, 则:

- 主干基础卷积层: $6 \times 4 = 24$
- 额外轻量残差块中的卷积层: $4 \times 2 = 8$
- 检测头卷积层: 2

总计约为:

$$24 + 8 + 2 = 34$$

个卷积层。

6.2.2 Stage C 的逐层张量流

Table 5: Stage C 的张量形状流

模块	输出形状	说明
Input	[B, 3, 320, 320]	resize 后RGB 图像
Stage 1 blocks	[B, 32, 320, 320]	4 个ConvBNAct
Pool 1	[B, 32, 160, 160]	下采样
Stage 2 blocks	[B, 64, 160, 160]	4 个ConvBNAct
Pool 2	[B, 64, 80, 80]	下采样
Stage 3 + residual	[B, 128, 80, 80]	4 个ConvBNAct + 1 个残差块
Pool 3	[B, 128, 40, 40]	下采样
Stage 4 + residual	[B, 256, 40, 40]	4 个ConvBNAct + 1 个残差块
Pool 4	[B, 256, 20, 20]	下采样
Stage 5 + residual	[B, 384, 20, 20]	4 个ConvBNAct + 1 个残差块
Pool 5	[B, 384, 10, 10]	下采样
Stage 6 + residual	[B, 512, 10, 10]	4 个ConvBNAct + 1 个残差块
Head conv	[B, 256, 10, 10]	检测头3x3 conv
Raw pred map	[B, 255, 10, 10]	三框输出通道
Final output	[B, 10, 10, 255]	grid-major 预测张量

6.2.3 检测头与输出

Stage C 仍然是单尺度检测头，但每个cell 预测三个框。因此输出维度从：

$$1 \times (80 + 5) = 85$$

变成：

$$3 \times (80 + 5) = 255$$

也就是最终输出：

$$[B, 10, 10, 255]$$

在loss 中再reshape 为：

$$[B, 10, 10, 3, 85]$$

6.3 Stage C 的检测方法、anchors 与target 编码

6.3.1 三组anchors

Stage C 使用的anchors 来自对训练集框尺寸的拟合，分别为：

$$a_1 = (0.052141, 0.076305) \tag{5}$$

$$a_2 = (0.199926, 0.288289) \tag{6}$$

$$a_3 = (0.609263, 0.649287) \tag{7}$$

这些anchor 宽高都是相对于整张图像归一化后的数值。

6.3.2 target 分配方法

当前三框target 编码采用如下策略：

- 先计算GT 框与三组anchor 的width-height IoU；
- 每个GT 只选择一个best_anchor 作为正样本；
- anchor IoU 高于阈值0.5 的其他anchor，从负样本中忽略；
- 同时记录collision_count，用于观察同一cell 的目标冲突情况。

因此Stage C 的监督张量包括：

- target_box[G,G,A,4]
- target_obj[G,G,A]
- target_cls[G,G,A,C]
- object_mask[G,G,A]
- noobj_mask[G,G,A]

6.3.3 预测解码与推理

当前推理解码逻辑为:

- 对 t_x, t_y, t_w, t_h 全部做sigmoid;
- t_x, t_y 表示cell 内偏移;
- t_w, t_h 与对应anchor 相乘, 得到预测框宽高;
- objectness 与类别置信度相乘得到最终分数;
- 再执行class-wise NMS。

6.4 Stage C 的loss 设计

Stage C 继续沿用完整YOLO-style loss 的三项分解:

$$L_{box} = 1 - \text{GIoU} \quad (8)$$

$$L_{obj} = L_{obj}^+ + \lambda_{noobj} L_{obj}^- \quad (9)$$

$$L_{cls} = \text{BCEWithLogits} \quad (10)$$

总损失仍然是:

$$L = \lambda_{box} L_{box} + \lambda_{obj} L_{obj} + \lambda_{cls} L_{cls}$$

超参数保持:

$$\lambda_{box} = 5.0, \quad \lambda_{obj} = 1.0, \quad \lambda_{noobj} = 0.5, \quad \lambda_{cls} = 1.0$$

6.5 Stage C 正式超参数

Table 6: Stage C 的核心超参数

配置项	当前设置
model name	deep_residual
depth_mult	2.0
use_residual	true
num_boxes	3
anchors	(0.052141, 0.076305), (0.199926, 0.288289), (0.609263, 0.649287)
anchor_ignore_iou	0.5
image size	320
grid size	10
classes	80
batch size	256
epochs	20
optimizer	AdamW
lr / min lr	1e-3 / 1e-5
scheduler	cosine
workers	64
weight decay	1e-4

6.6 Stage C 结果

Table 7: Stage C: deep_residual + three-box full loss

项目	数值
Run ID	deep_residual_three_box_full_loss_20260419_043041
Params	26,560,607
输出张量形状	(1, 10, 10, 255)
最优epoch	13
最优验证总损失	2.956484
最终训练总损失	2.566497
最终验证总损失	3.202718

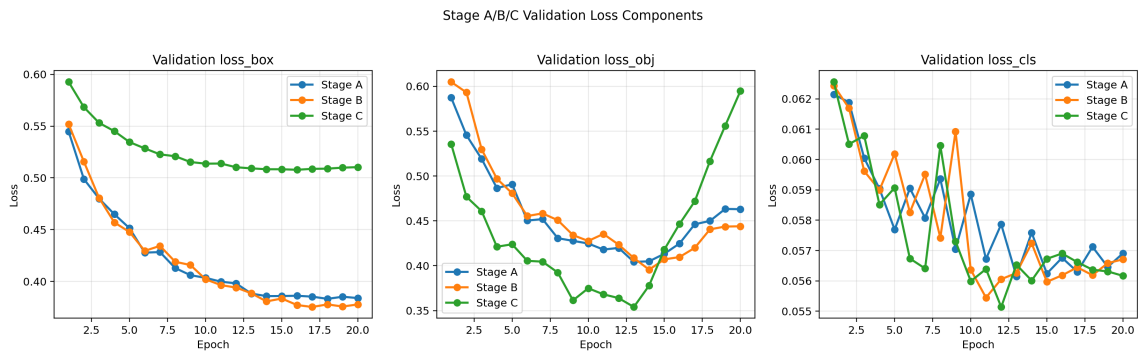


Figure 3: Stage A、B、C 的验证集loss_box、loss_obj、loss_cls 曲线。

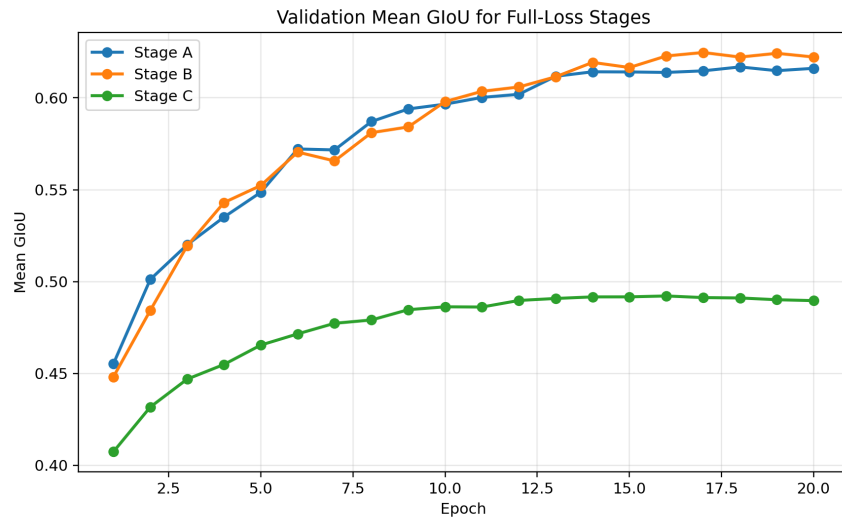


Figure 4: Stage A、B、C 的验证集mean_giou 曲线。

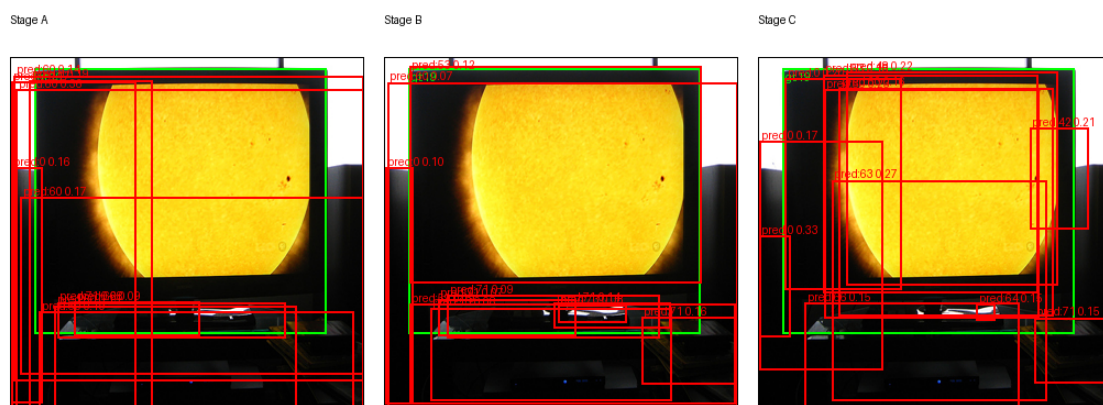


Figure 5: 同一验证样本在Stage A、Stage B、Stage C 下的预测可视化。绿色框为真实框，红色框为预测框。

6.7 Stage C 结论与转移原因

Stage C 已经完成了以下工程目标：

- 三框检测头已经实现；
- 三框target 编码已经实现；
- best-anchor 匹配和ignore 机制已经实现；
- 三框detector 可以full training；
- 可以生成GT 与预测框对比图像。

但是从验证损失看：

- Stage C 没有超过Stage B；
- 当前最优结果仍然是deep_residual + single-box full loss。

因此从Stage C 向下一部分转移的原因是：

仅看loss 已不足以判断三框系统的真实检测能力，需要补充更正式的检测指标与效率指标。

7 第五阶段：检测评估补充与COCO 子集官方风格评测

7.1 为什么需要评估补充

截至Stage C，我们已经有：

- train / val total loss
- loss_box、loss_obj、loss_cls
- GT vs Pred 可视化

但checkpoint 6 最终需要的不只是loss 比较，还需要：

- 参数量

- FLOPs
- 推理FPS
- 更正式的检测指标

因此，我们补充了：

- 工程内部评估脚本；
- 基于pycocotools 的COCO 子集官方风格评测；
- FLOPs 和FPS 的测量。

7.2 为什么官方风格评测只在COCO 子集上进行

Unified 的验证集all_val.jsonl 混合了：

- voc2012
- coco2017

这两部分并不共享一个严格统一的语义label space，因此不能在混合集上直接做语义上严格的官方COCO 风格评测。

因此，当前更严谨的做法是只在：

- /home/lidz/YOLO/DataSet/Unified/manifests/coco2017_val.jsonl

上使用pycocotools 进行COCO 子集评测。

7.3 Stage A/B/C 在COCO 子集上的结果

Table 8: Stage A/B/C 的COCO 子集官方风格评测结果

Model	Boxes/cell	Params	GFLOPs	FPS	AP50	AR@100
Stage A	1	17.66M	31.56	140.50	0.000486	0.001081
Stage B	1	26.52M	42.18	113.28	0.000588	0.001211
Stage C	3	26.56M	42.19	116.44	0.000122	0.000999

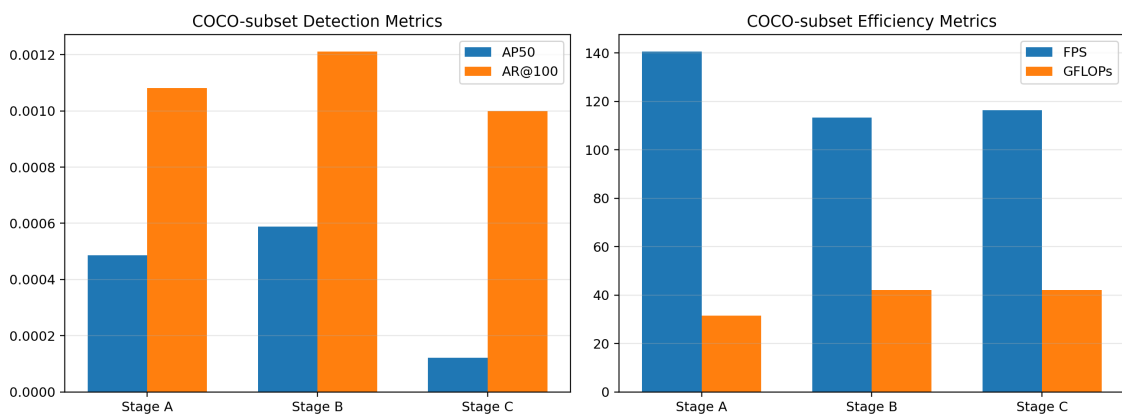


Figure 6: Stage A/B/C 在COCO 子集上的检测指标与效率指标对比。

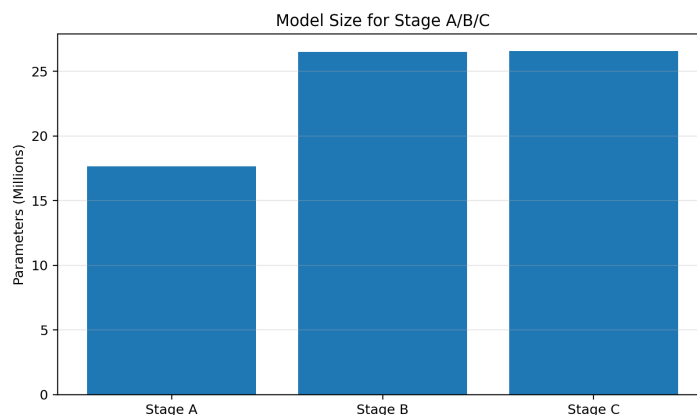


Figure 7: Stage A/B/C 的参数量对比。

7.4 评估补充阶段结论

这一阶段得到两个重要结论：

1. 内部val loss 的排序与COCO 子集官方风格评测的排序是一致的；
2. Stage C 的问题不是内部loss 指标的假象，它在更正式的COCO 子集评测下同样没有超过Stage B。

更具体地说：

- Stage B (`deep_residual + single-box full loss`) 仍然是当前最强的实用系统；
- Stage C 已经实现并且可以full-run，也可以被正式评测，但在当前匹配和监督设计下仍然较弱；
- 所有模型的绝对AP 值仍然很低，这说明当前detector family 仍然属于早期baseline，而不是成熟检测器。

8 当前loss 设计、网络演化方向与阶段性总括

8.1 已经完成的演化路径

截至目前，checkpoint 6 已经完成的路径可以概括为：

1. 在简化MSE loss 下完成Backbone 的受控对比；
2. 在该阶段选择`deep_cnn` 作为更合适的主干；
3. 将loss 升级为完整的单框YOLO-style 版本；
4. 在完整单框loss 下比较残差结构；
5. 在最优单框残差基线之上实现三框检测头；
6. 补充更正式的检测评估与效率评估。

8.2 loss 设计从早期版本到当前版本的变化

项目在checkpoint 6 开始时使用的是：

$$L = \lambda_{cls} L_{cls}^{MSE} + \lambda_{box} L_{box}^{MSE}$$

这是一种更接近教学用途的简化检测loss。

当前已经完成并正式使用的loss 则是：

$$L = \lambda_{box} L_{box}^{GIoU} + \lambda_{obj} L_{obj}^{BCE} + \lambda_{cls} L_{cls}^{BCE}$$

同时：

- 单框输出从84 升级到85；
- 三框输出从85 扩展到255；
- target 从简单的target_cls + target_box + object_mask 扩展为包含target_obj、noobj_mask 和collision_count 的更完整结构；
- 三框版本进一步引入了anchors、best-anchor 分配与ignore 机制。

8.3 当前最优实用系统

从当前已经完成的全部实验看，最优实用结果是：

`deep_residual + single-box full loss`

原因在于：

- 它在Stage A/B/C 中取得了最低的验证总损失；
- 它在COCO 子集官方风格评测中也取得了最高的AP 与AP50；
- 它比当前Stage C 三框系统更稳定，也更容易解释。

8.4 当前网络演化方向

到目前为止，网络演化方向已经不再是继续盲目堆新的Backbone，而是转向：

- 先冻结当前最优Backbone；
- 重点改进Stage C 三框系统的匹配、ignore、objectness 和box 参数化；
- 等三框系统真正达到或超过当前单框残差基线之后，再考虑更大系统级升级，如更细网格、neck 或多尺度输出。

9 总结

截至本报告写作时，checkpoint 6 已经完成了两次关键跨越：

1. 从“在简化MSE loss 下比较Backbone”推进到“在完整YOLO-style loss 下比较完整检测系统”；
2. 从只看train/val loss 推进到同时使用参数量、FLOPs、FPS 和COCO 子集官方风格评测。

虽然当前detector 在绝对AP 上仍然很弱，但这并不意味着工作无效。相反，我们已经得到了一套正式、可复现、可比较的工程基线，它现在具备：

- 配置驱动的full 训练；
- 每轮正式实验的结果文本、配置快照和元数据；
- 可视化产物；
- loss 曲线；
- 参数量；
- FLOPs；
- FPS；
- COCO 子集上的官方风格评测。

因此，这一版报告已经足以作为checkpoint 6 的第一版正式阶段总结，同时也为后续继续改进Stage C 提供了稳定的比较基线和分析依据。